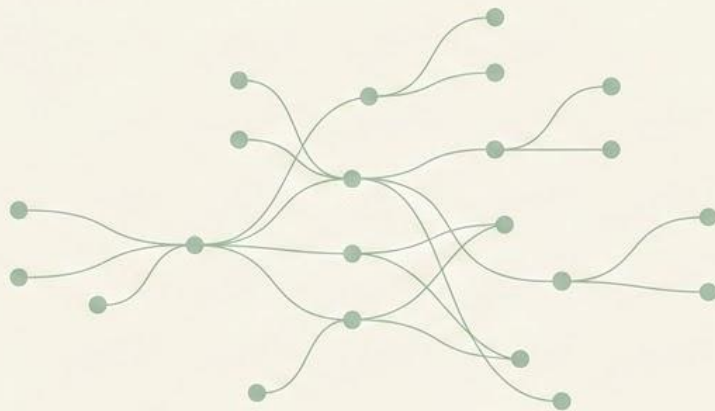




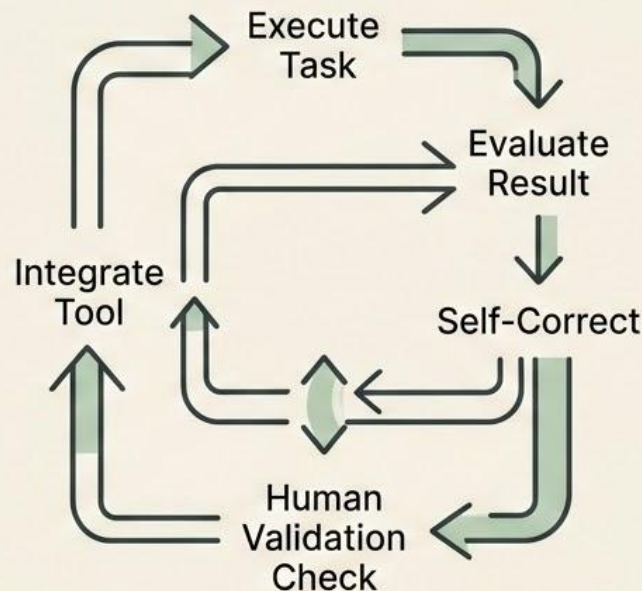
Building Stateful AI Agents with LangGraph

Beyond Linear AI Chains

Traditional chains are linear: Standard DAG frameworks struggle when workflows need dynamic backtracking or iteration.

The requirement for loops: Autonomous agents need cycles to self-correct, execute tools iteratively, or seek human validation.

Cyclic native execution: LangGraph is built from the ground up to handle complex cyclic pathways smoothly.



The Pillars of LangGraph



Nodes

Python functions that execute computation, such as calling an LLM, executing a tool, or formatting data.



Edges

Control flow pathways that connect nodes, determining static transitions or dynamic conditional routing.



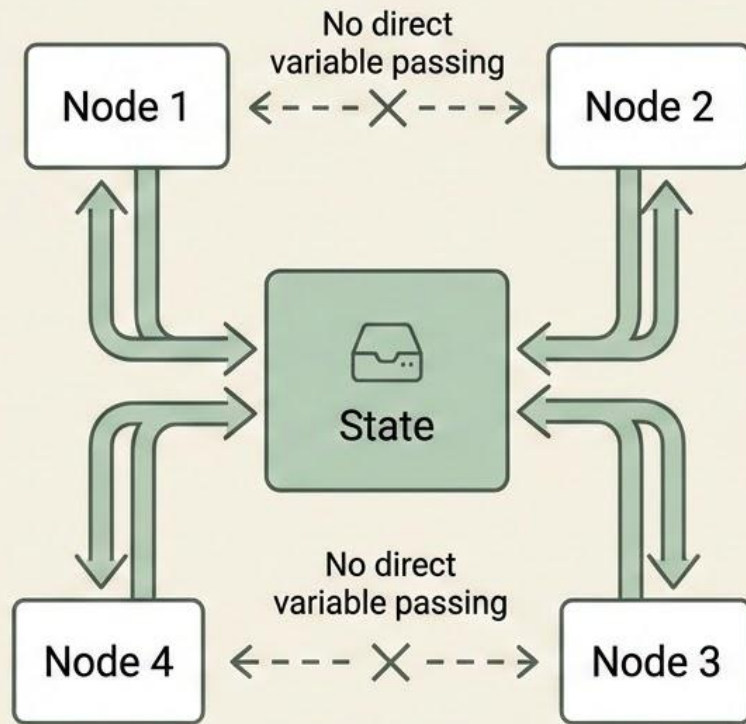
State

A centralized, shared database acting as the common memory passport accessible and mutable by all nodes.

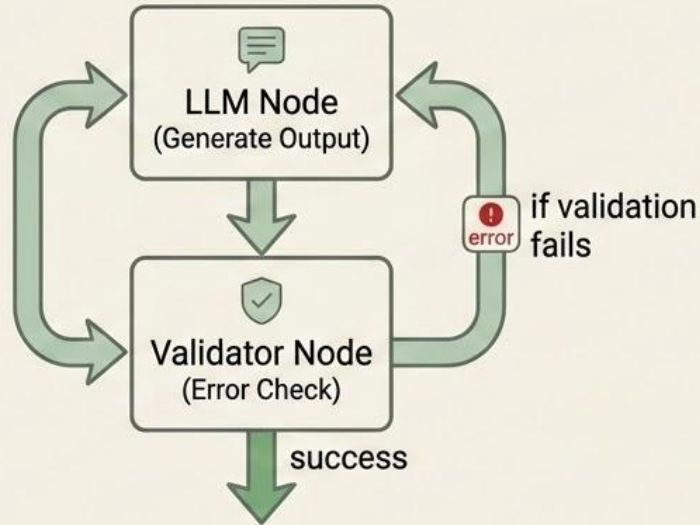
Centralized State Memory

The Unified Passport: Nodes do not pass variables directly to each other. They read from and write updates to a shared central State.

Decoupled Logic: Makes complex agent workflows highly modular, easy to isolate, and incredibly resilient to test.



Iterative Cycles & Self-Correction



If validation fails, loop back.

LLM Node generates structured output -> Validator Node detects a schema error -> Workflow dynamically routes back to the LLM Node with error details for self-correction.

Transitions & Dynamic Routing



Static Edges: Permanent, deterministic transitions (e.g., Node A always moves directly to Node B).



Conditional Edges: Dynamic router functions that inspect the current State and choose the next node on the fly.



Start & End Anchors: Built-in entry and exit boundaries mapping compilation starting and final endpoints.

State Reducer Functions

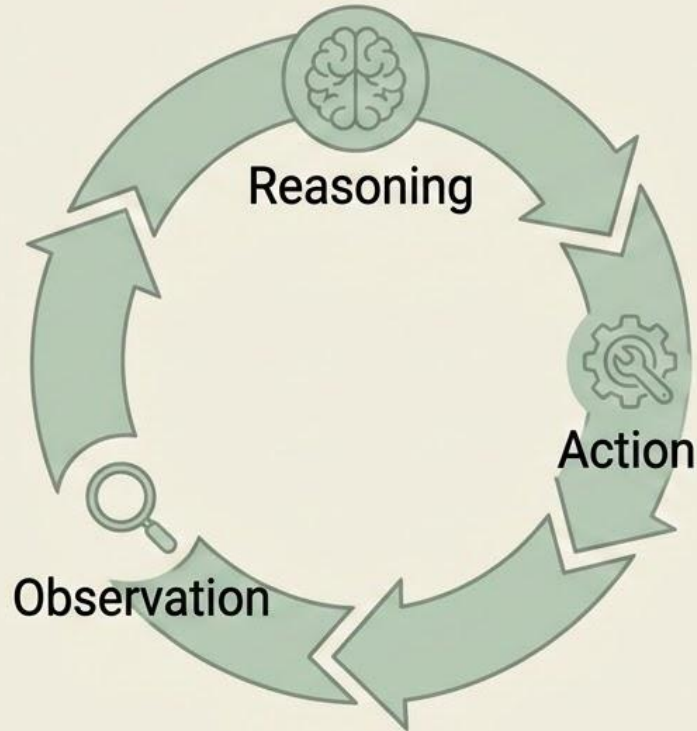
Default Overwrite:

Returning a key updates its value by replacing it entirely. Great for state flags, user queries, and simple step indices.

Accumulator / Append:

Using special annotations like `Annotated[list, add]` tells LangGraph to append list items rather than overwrite—crucial for building chat histories.

The ReAct Agent Pattern



The fundamental structural loop behind modern autonomous AI.

The Loop: Agent node analyzes a query → Agent demands a tool call → Tool node executes the tool → Agent reviews the outcome → Loop terminates only when the goal is achieved.

Coding Step

Linear Graphs —
Creating a basic,
standard step-by-
step pipeline.

1



2

State & Reducers —
Implementing an
appending chat
memory state.

ReAct Loop —
Hooking up LLMs,
active tools, and
conditional routing.

3



4

Live Run & Trace —
Testing and
visualizing agent
execution.

Let's Write Some Code!



We are going to build a fully functional,
cycle-based agent from scratch right now.